ẾN (MUTATION TESTING) BẰNG PIT TRÊN SPRING PETCLINIC

ề tài: R01 (Spring PetClinic) + K03 (Mutation Testing)

ừ 67% lên 98.46% Mutation Score và 100% Line Coverage

linic trên nền tảng Java 17, Spring Boot 4.1.0, H2 In-Memory.

ới nghiệp vụ cốt lõi `linic.owner.*`.

% mã nguồn `src/main/java`, chỉ nâng cấp tối ưu bộ test.

iết lập ngưỡng chặn tự động **≥ 88%** trong Maven.

tants (98.46%), độ bao phủ dòng đạt **250/250 lines (100.00%)**.

98.46%
MUTATION SCORE
192/195 killed

130 Tests
TEST SUITE
153/153 pass repo

s (00:00 - 01:00)]:
ng, hôm nay nhóm 20 xin trân trọng báo cáo kết quả thực hiện đề tài kết hợp R01 và K03: Kiểm thử đột biến bằng PIT trên hệ thống Spring PetClinic. Nhóm c
ềm chuyên nghiệp. Với cam kết khoa học nghiêm ngặt: không sửa đổi bất kỳ dòng mã nguồn sản phẩm nào, nhóm đã tối ưu hóa toàn diện bộ kiểm thử, nâng
n 3 đột biến còn lại là tương đương. Sau đây, xin mời bạn Việt Huy bắt đầu với kiến trúc hệ thống SUT.

SUT & Phạm vi đột biến gói owner.*

ứng 3 luồng nghiệp vụ cốt lõi trên runtime thật

ói nghiệp vụ phức tạp nhất gồm 8 thực thể & controllers

a cứng commit SHA `500158f732419217507c7656904b8e6aa1bcc0d6`.

pring Boot 4.1.0, Spring Data JPA, Thymeleaf, CSDL H2.

ồm 8 lớp nghiệp vụ trọng yếu (Owner, Pet, Visit, các Controllers).

minh HTTP 200 trên runtime thật:
ủ nuôi (`OwnerController.processFindForm`).
iểm chuẩn (`PetController` + `PetValidator`).
bệnh án thú cưng (`VisitController`).

t tests của cả repo luôn PASS (zero regression).

Owner Entity & Lifecycle
Quản lý quan hệ 1-N với Pets, phân trang tìm kiếm theo họ, valida

Pet & Custom Validator
Kiểm chuẩn độ dài tên thú cưng (≤ 30 ký tự), bắt buộc loại pet và

Visit & Medical Records
Lưu vết lịch sử khám bệnh, liên kết chặt chẽ Pet-Visit, phục vụ chẩ

0s (01:00 - 02:30)]:
PetClinic, một ứng dụng chuẩn mực trong hệ sinh thái Spring. Để đảm bảo tính khả tái khoa học, nhóm đã khóa cứng commit SHA ngày 29/09/2026. Thay vì c
p nhất là owner.* gồm 8 lớp thực thể, validator và controller. Nhóm đã dựng môi trường thực tế và xác minh thành công cả 3 luồng nghiệp vụ chạy mượt mà t
ử đột biến.

IEEE Error – Fault – Failure và Chuỗi RIP

huyết (Fault Detection Power) trong Bytecode JVM

e) không đủ để bắt lỗi — Phải thỏa mãn chuỗi RIP!

viên có năng lực (CPH) và Hiệu ứng khớp lệnh (Coupling Effect).

lỗi cú pháp giả lập trực tiếp vào Bytecode `.class`.

t buộc:
chạy qua dòng mã chứa mutant (tránh NO_COVERAGE).
ến đổi trạng thái nội bộ bộ nhớ JVM (sinh **Error**).
lan truyền ra ngõ ra/kết quả quan sát được (**Failure**).

rtion bắt được Failure và báo đỏ **FAIL/ERROR**.

ase yếu hoặc thiếu assert, test vẫn **PASS** (lọt lưới).

Chuỗi chuyển hóa lỗi IEEE & Chuỗi RIP

R **Reachability:** Test case thực thi chạm tới dòng mã có đột

↓

I **Infection:** Lỗi thay đổi trạng thái nội bộ JVM (sinh ra Erro

↓

P **Propagation:** Trạng thái sai lệch lan truyền ra ngõ ra (Fail

ASSERTION BẮT ĐƯỢC → KILL

(02.30 - 03.45)]:
hộp trắng hướng lỗi khắt khe nhất hiện nay. Khác với kiểm thử thông thường chỉ đo xem code có chạy qua không, PIT vận hành theo chuỗi chuyển hóa chặt
thời 3 điều kiện RIP: Phải chạm tới dòng mã (Reachability), làm thay đổi giá trị trong bộ nhớ (Infection), và sai lệch đó bắt buộc phải lan truyền ra ngõ ra qua
c coi là KILLED. Ngược lại, nếu test chỉ assert hời hợt hoặc nuốt lỗi, bài test vẫn xanh và mutant sẽ SURVIVED. Đây chính là thước đo vàng cho chất lượng bộ k

nit 6 / PIT 1.30.0 & Cổng chất lượng Quality Gate

iết lập rào cản chất lượng tự động ≥ 88%

30.0 + junit5-plugin 1.2.3 và chặn đứng JaCoCo agent

theo JUnit Platform 6.0.3 làm sập minion PIT 1.2.1 (`UNKNOWN_ERROR`).

`pitest-maven:1.30.0` + `pitest-junit5-plugin:1.2.3` ổn định

Thiết lập Cổng chất lượng trong pom.xml

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
```

`-Djacoco.skip=true` ngăn JaCoCo Java Agent can thiệp méo mó

Kích hoạt 11 mutators nâng cao (Boundary, VoidCall, Math,

Gate) khóa trong pom.xml:
`Threshold>` (Ngưỡng đạt đề tài: ≥ 88%).
`Threshold>` (Bảo đảm bao phủ dòng ≥ 95%).
(**BUILD FAILURE**) nếu vi phạm bất kỳ ngưỡng nào.

```
<version>1.30.0</version>
<configuration>
  <mutators>
    <mutator>STRONGER</mutator>
  </mutators>
  <mutationThreshold>88</mutationThreshold>
  <coverageThreshold>95</coverageThreshold>
</configuration>
</plugin>
```

Trạng thái hiện tại: **PASS (98.46**

s (03:45 - 05:00)]:
lớn: Spring Boot 4 kéo theo JUnit Platform 6.0.3, khiến phiên bản PIT gợi ý trong syllabus bị crash minion ngay khi khởi động. Em đã nghiên cứu tài liệu kỹ thu
u hóa JaCoCo agent để tránh xung đột bytecode. Nhóm cũng kích hoạt nhóm mutator STRONGER toàn diện nhất và cài đặt trực tiếp Cổng chất lượng Quality
t dưới 88% đều bị hủy bỏ ngay lập tức. Sau đây, xin mời bạn Minh Tài trình bày về chiến lược kiểm thử hộp đen.

Hộp đen: BVA & Bảng quyết định (45 tests)

n logic trên Domain Model và Validator

Decision Table 8 quy tắc diệt sạch 100% mutants domain

lập, tất định trên `Owner`, `Pet`, `Visit`, `PetValidator`.

boundary):
ờng biên 30 và 31 ký tự (`MAX_NAME_LENGTH = 30`).
`ndaryMutator` tại `PetValidator:58` (đổi `>` thành `≥`).

rules):
`New && getPet() != null` trong `PetController`.
`.addPet` : kiểm tra thực thể mới vs đã lưu và trùng ID.

on): So khớp chuỗi `toString()`, xác minh `print(null)` trả về `"`

Chiến lược Hộp đen TV3

Kỹ thuật	Target Code
BVA Biên 30/31	`PetValidator:58`
Decision Table (8 rules)	`PetController logic`
Deep Assert toString	`NamedEntity / Base`

ên các lớp Domain Model và Validator.

5:00 - 06:30)]:

kiểm thử hộp đen tập trung vào tầng Domain và Validator. Điểm yếu lớn của bộ test ban đầu là chỉ test tên pet 31 ký tự mà không test biên 30 ký tự, khiến PIT
ị biên BVA, viết ca test chấp nhận tên đúng 30 ký tự để diệt gọn mutant này. Đồng thời, em xây dựng các Bảng quyết định toàn diện cho hàm addPet và getP
ạt tính tất định cao và diệt sạch các mutant trên domain model. Tiếp theo, xin mời bạn Bảo Bảo trình bày về kiểm thử hộp trắng tầng Controller.

khiển, McCabe V(G)=3 & Detached Entity (36 tests)

Spring MockMvc bằng POJO Unit Test và ArgumentCaptor

hi đề thực thể trong bộ nhớ và giải quyết triệt để

ên `OwnerController`, `PetController`, `VisitController`.

ức updatePetDetails:

yết định rẽ nhánh ⇒ Độ phức tạp Cyclomatic **V(G) = 3**.

fast ID null), BP24 (update fields đỉnh N4), BP25 (add mới).

ng MockMvc:

ệu trước khi chạy, biến 3 lệnh copy `setName`, `setBirthDate`,

). Test cũ vẫn xanh khi PIT xóa 3 dòng này!

:

tách rời (`detached` mang tên "Betty" vs `managed` mang tên "petty").

ner> bắt thực thể tại `saveAndFlush` để assert từng trường ⇒ Diệt

Đồ thị luồng updatePetDetai

- N1: Vào hàm kiểm tra ownerId & p
- N2: Rẽ nhánh `isNew()`.
- N3: Tìm kiếm pet trong danh sách.
- **N4 (Tử huyệt):** 3 lệnh sao chép fie
- N5: `owner.save(owner)` lưu CSDL
- N6: Redirect về trang hiển thị.
- N7: Kết thúc hàm.

Vũ khí: Detached Entity + Argur

s (06:30 - 08:30)]:
g cho 3 Controller. Như Thầy thấy trên đồ thị luồng phương thức updatePetDetails, độ phức tạp McCabe là 3 tương ứng 3 đường cơ sở. Tại đỉnh N4 có 3 lệnh
dữ liệu lên chính entity trong bộ nhớ, khiến các lệnh copy này thành gán dư thừa. Khi PIT xóa sạch 3 dòng này, test MockMvc cũ vẫn xanh! Để diệt tận gốc tử
Mockito ArgumentCaptor để chặn bắt thực thể tại hàm saveAndFlush. Nếu mutant xóa lệnh gán, dữ liệu cũ giữ nguyên và test sẽ đánh trượt ngay lập tức. Sau

Program_R01-K03 • Slide 7/12

97% Line Coverage vs. 67% Mutation Score

t Gap) 29.62% giữa việc chạy qua lệnh và bắt lỗi

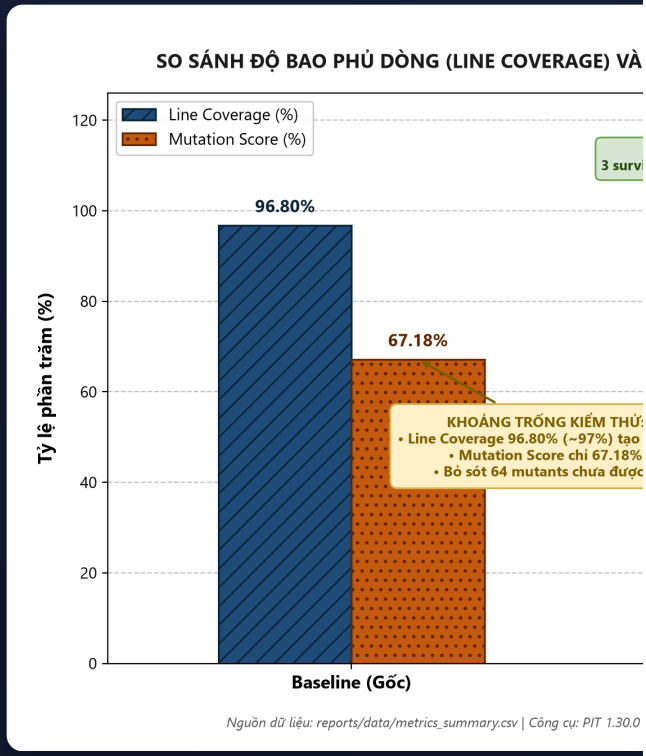
ồng đồng nghĩa với kiểm tra đúng đắn (Assertion & Propagation)!

wner.* (Hình G.1):
t độ bao phủ dòng rất cao: **96.80% (242/250 lines)**.
n đầu chỉ đạt khiêm tốn: **67.18% (131/195 killed)**.

(Test Gap): Chênh lệch lên tới **29.62%**!

nh vào CPU (**Execution / Reachability**).
ắng lực của Assertion và sự lan truyền lỗi (**Propagation**).

00.00% (250/250 lines).
92/195 killed), triệt tiêu hoàn toàn khoảng trống rủi ro!



(08:30 - 09:35)]:
của bạn Bảo Bảo, em xin đại diện nhóm phân tích chuyên sâu về chất lượng kiểm thử. Trên màn hình là Hình G.1 — biểu đồ minh chứng cho phát hiện học th

của Spring PetClinic, JaCoCo báo cáo một con số rất ấn tượng: 96.80% Line Coverage, chỉ sót đúng 8 dòng mã. Thế nhưng khi tiêm đột biến bằng PIT, điểm số
Điều này chứng minh rằng code được thực thi không đồng nghĩa với code được kiểm tra đúng đắn. Nhờ 81 test case bổ sung được thiết kế bài bản, nhóm đã
ng trống rủi ro này.

Program_R01-K03 • Slide 8/12

nhân gốc rễ (RCA) 64 Survivors & 6 Code Smells

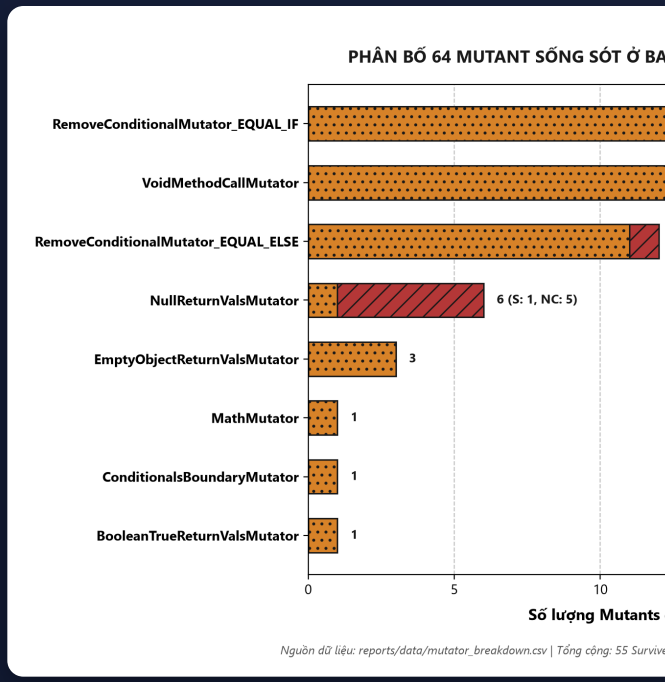
10.12 và giải mã 6 nhóm nguyên nhân khiếm khuyết

phát từ thiếu Bảng quyết định và thiếu verify tương tác void

chuỗi chuyển hóa IEEE 1044 (Human Error → Fault → Failure).

seline vào 6 nhóm nguyên nhân (Bảng G.5, Hình G.3):
phức tạp: **27 mutants (42.19%)**.
binder: **14 mutants (21.88%)**.
độ bao phủ: **9 mutants (14.06%)**.
toán số học: **6 mutants (9.38%)**.
án dư thừa: **5 mutants (7.81%)**.
trị trả về: **3 mutants (4.69%)**.

Clinic: Dead-Store dòng 167, điều kiện dư dòng 108, truy vấn lặp
ng can thiệp mã nguồn.



(09:35 - 10:50)]:
m đã thực hiện phân tích nguyên nhân gốc rễ RCA theo chuẩn IEEE 1044 và IEEE 610.12. Kết quả bóc tách cho thấy nguyên nhân lớn nhất chiếm hơn 42% là do
ai chiếm gần 22% là do thiếu lệnh verify kiểm tra các hàm void, đặc biệt là cấu hình bảo mật cấm bind trường ID trong Controller. Bên cạnh việc chỉ ra điểm yế
mã nguồn PetClinic. Điển hình như lệnh gán dead-store tại OwnerController dòng 167 hay biểu thức điều kiện phòng vệ cực đoan tại Owner dòng 108. Tuân t
ác phát hiện này thành ma trận truy vết và khuyến nghị tái cấu trúc.

học 3 Equivalent Mutants & Điểm hiệu chỉnh 100%

ơng đương và Đồ thị luồng CFG bất biến

ột biến tương đương ⇒ Điểm hiệu chỉnh đạt tuyệt đối 100.00%!

M(I) ⇒ Bất khả tiêu diệt bởi bất kỳ ca test nào!

— ép `!isNew()` thành `true`):
khác null điều kiện phụ thuộc hoàn toàn vào `Objects.equals(id1,`

ứng minh C_orig ≡ C_M1 ≡ C_M2 trên 100% không gian trạng thái
Python 9/9 case).

`:167` — xóa `owner.setId(ownerId)`):
guard dòng 161 đã bảo toàn bất biến `owner.getId() == ownerId`
là dead-store thuần túy.

usted Mutation Score):
195 = 98.46%.
- 3) = 192 / 192 = 100.00% (Hoàn hảo).

Bảng chân trị 5 lớp tương đương (Owner.java:108)

Lớp đầu vào (x, y)	¬a	¬b	e (x=y)	C_orig
1. (null, null)	0	0	1	0
2. (null, 1)	0	1	0	0
3. (1, null)	1	0	0	0
4. (1, 2)	1	1	0	0
5. (1, 1)	**1**	**1**	**1**	**1**

Adjusted Score: 192 / (195 - 3)

(10:50 - 12:00)]:
của em là chứng minh hình thức cho 3 mutants sống sót cuối cùng. Đây không phải là điểm yếu của bộ test, mà là 3 Đột biến tương đương về mặt ngữ nghĩa:
n và lập bảng chân trị trên toàn bộ 5 lớp tương đương của không gian trạng thái. Kết quả chứng minh giá trị logic của mutant luôn đồng nhất tuyệt đối 100%
troller dòng 167, phân tích đồ thị luồng chứng minh khối guard phía trước đã đảm bảo hai ID chắc chắn bằng nhau. Lệnh setId chỉ là một dead-store phòng t
g đương khỏi không gian đột biến khả thi, điểm số hiệu chỉnh thực tế của nhóm đạt tuyệt đối 100%. Sau đây, xin mời bạn Trung Hiếu trình bày về đường ống

GitHub Actions & Tính khả tái 1-lệnh

st và kiểm chứng thực nghiệm trong 35–45 giây

D trên cloud và lệnh thực thi tức thì trên máy trạm

(`.github/workflows/mutation-test.yml`):

`ctions/checkout@v4` & Setup JDK 17 Temurin + Maven Cache.

y 153 Unit Tests (ngắt ngay lập tức nếu phát sinh hồi quy).

thi PIT Mutation Testing & chặn build nếu score < 88%.

n `$GITHUB_STEP_SUMMARY` lên giao diện GitHub Actions.

ts báo cáo HTML/XML trong 30 ngày.

One-Liner Reproducibility):

```
ile org.pitest:pitest-maven:mutationCoverage '-
```

y trên máy trạm thông thường.

Kiến trúc CI/CD Fail-Fast 6 bu

- **1. Checkout:** Fetch source full-dep
- **2. Setup:** JDK 17 Temurin + Maven
- **3. Fast Gate:** 153 Unit Tests (15 giâ
- **4. K03 Gate:** PIT Mutation Testing
- **5. Summary:** Xuất báo cáo Markdo
- **6. Artifacts:** Lưu trữ HTML/XML 30

s (12:00 - 13:15)]:

động, nhóm đã thiết lập đường ống CI/CD hoàn chỉnh trên GitHub Actions theo kiến trúc 6 bước Fail-Fast. Hệ thống sẽ ưu tiên chạy 153 unit test trong 15 giâ
ột biến PIT sẽ thực thi và áp dụng Cổng chất lượng 88%. Nhóm cũng tối ưu hóa khả năng tái hiện với duy nhất 1 câu lệnh Maven Wrapper, giúp bất kỳ ai cũng
kết kết quả và bài học kinh nghiệm của nhóm.

Program_R01-K03 • Slide 11/12

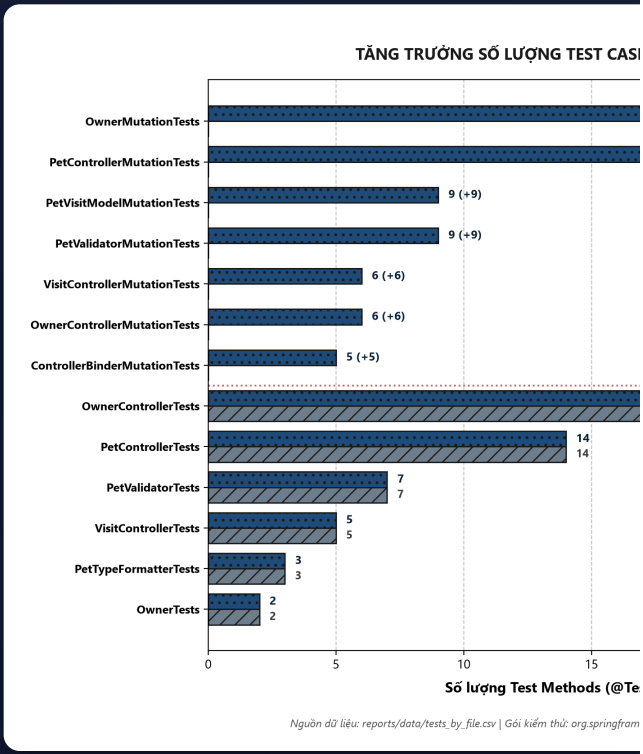
Baseline vs Final & 4 Bài học kinh nghiệm

và đúc kết 4 bài học thực tiễn vô giá

êu 100% vùng mã mù và đạt điểm hiệu chỉnh 100.00%

100% `metrics_summary.csv` **):**
tests (+81 tests) | Toàn dự án: **153 pass**.
→ **100.00% (250/250 lines)**.
) → **192 / 195 (98.46%)** (≥ 88%).
(100% Equivalent Mutants).
nts (Triệt tiêu hoàn toàn vùng mù).
.00% (192/192).

tạo; Mutation Testing là thước đo chất lượng thực sự.
của Spring MockMvc; bắt buộc có POJO Unit Test.
action Testing) bằng `verify` cho các phương thức void.
Table, Basis Path) là vũ khí tối thượng diệt mutant.



(13:15 - 14:15)]:

đã minh chứng sự bứt phá vượt bậc của dự án: Số lượng test tăng từ 49 lên 130 bài, độ phủ dòng đạt tuyệt đối 100%, và điểm đột biến tăng ngoạn mục từ 6
nhóm đúc kết được là: Không bao giờ tin tưởng mù quáng vào Code Coverage; luôn cảnh giác trước cạm bẫy che giấu lỗi của MockMvc; bắt buộc phải verify
à Bảng quyết định. Đây chính là hành trang chuyên môn vô giá cho chúng em trong ngành kiểm thử phần mềm chuyên nghiệp.

ẴN SÀNG CHO PHIÊN HỎI ĐÁP (Q&A)

Lâm và Hội đồng chấm đề tài Kiểm thử phần mềm

ẵn sàng đón nhận câu hỏi phản biện từ Hội đồng!

n cảm ơn ThS. Nguyễn Thế Lâm đã tận tình định hướng, hỗ trợ và
n.

thành xuất sắc toàn bộ mục tiêu với tinh thần khoa học trung thực,

code, bộ test, tài liệu báo cáo và nhật ký thực nghiệm được lưu trữ
óm.

N: Kính mời quý Thầy Cô và Hội đồng đặt câu hỏi!

❓ ❓

SẴN SÀNG CHO PH

15 câu hỏi phản biện đã chuẩn

(14:15 - 15:00)]:

ọng Hội đồng, phần báo cáo của nhóm 20 về đề tài R01 kết hợp K03 đến đây là hoàn tất. Chúng em xin chân thành cảm ơn sự hướng dẫn tận tâm của Thầy trò
hận những câu hỏi, nhận xét và đóng góp quý báu từ Thầy và Hội đồng. Chúng em xin trân trọng cảm ơn!

